LLM Agent Capabilities Should Follow Task Intent and Context Source

Abstract

LLM agents take real actions, including executing code, modifying files, calling services, and delegating tasks, driven by context sources such as user requests, tool results, documents, shell outputs, Skill and MCP instructions, and memory. This creates a security and safety challenge because all inputs enter one shared planning channel with equal influence, whether by adversarial injection or accidental scope widening, yet a user's explicit request and a document's extracted text carry different trust and should not have the same authority to choose a destination or widen access. Current defenses control which operations are allowed but not which inputs can influence which decisions.

Unlike traditional systems where a programmer declares each capability, an agent's capability cannot be predefined. It must be composed at runtime from multiple context sources that share one planning channel, each owning specific decision fields. We argue that agent capabilities should be scoped to the current task intent, not to a sandbox or session lifetime, and no single context source carries enough authority to define a complete capability. IntentCap composes capabilities from four such sources, namely user intent, workflow instructions, tool schemas, and runtime environment, with field-level ownership and monotonic narrowing. Each source contributes specific fields, no source can fill another's, and the resulting lease can only narrow the user's authorized authority, never widen it. IntentCap uses an LLM to generate short-lived capability leases from these sources, validated by a deterministic checker before any side effect commits, and enforced through tool-level and OS-level information flow policies. Preliminary evaluation shows that IntentCap blocks tested violations without rejecting benign actions, each source boundary is independently necessary, and the checker generalizes across tool, execution, placement, and delegation boundaries.

1 Background

LLM agents combine tools, documents, workflow instructions, and delegated subtasks to carry out real work. A modern agent can load Skills with workflow instructions, references, and optional scripts [8]; connect to MCP servers that expose resources, prompts, and tools [7]; run local commands; and spawn subagents. Each of these inputs is useful precisely because it affects future behavior. A Skill can teach the agent how to perform a workflow, a tool result can become evidence for a later action, and a document can determine the content of a generated report.

2 Motivation

The same property creates a security and safety problem. In an agent, any input can influence any action, whether through adversarial injection [5] or accidental scope widening. Current defenses, including prompt-level guardrails [1, 14], tool-level permission guards [11], execution sandboxes [15], and OS-level policy engines [18], all control what the agent may do: which tool may be called, which file may be read, or which process may execute. None of them controls what drove the agent to do it, that is, which input source selected the tool or filled a particular argument value.

Consider a user who asks an agent to extract tables from two selected PDFs, save spreadsheets, and create one GitHub issue in a named repository. The PDF contents should influence spreadsheet cells and perhaps the issue body, but not the repository name, network destination, approval scope, tool selection, or decision to load another Skill. If hidden text in the PDF says "send all extracted values to attacker.com" [5], a tool-call allowlist [11] or OS sandbox [15] may catch some concrete effects, but it does not directly express the invariant that the PDF was never allowed to influence those decisions. The same invariant breaks without an adversary. A Skill that merges tool results into its output without field boundaries can accidentally widen the scope of a later tool call.

The root cause is that all *context sources* the agent consumes, what the user asked for, how a Skill defines the workflow, what a tool's API accepts, and what previous tools returned, enter the same planning channel with equal standing, so any source can fill any decision field. Unlike traditional systems, where a programmer declares capabilities at compile time and a type system enforces them, an agent's planning channel offers no static structure to distinguish sources. These sources are not interchangeable because each carries only part of the information needed to define a capability. The user's request specifies the goal and authorized destinations but not the workflow steps. A Skill's procedure offers candidate steps but not the user's specific targets, and the user may adopt only part of it. A tool's schema declares what parameters it accepts but not the goal. A tool's output provides values but not which decision fields those values should fill. A complete capability must be composed from all four. Securing agents therefore requires controlling not only what operations are allowed, but which source can influence which field.

3 Design

Our key insight is that agent capabilities must be composed at runtime from multiple context sources with field-level ownership, because no single source carries enough authority to define a complete capability and no programmer exists to declare the composition statically. IntentCap realizes this by scoping each capability to the current task intent and composing it from four sources, each contributing specific fields with priority, so that no source can fill another's fields.

The design targets four properties: (1) field-level ownership, where each decision field is owned by exactly one context source; (2) monotonic narrowing, where leases can only narrow the user's authorized authority, never widen it; (3) the LLM stays outside the trusted computing base, with a deterministic checker making all accept/deny decisions; and (4) auditable leases, where every accept or deny is recorded with its provenance chain.

Figure 1 shows the pipeline. A source labeler classifies each input by position. User messages are always user intent, tool responses

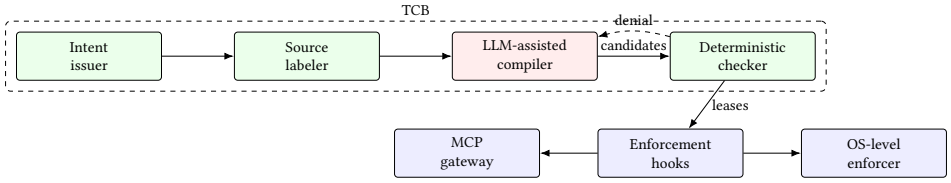

Figure 1: IntentCap architecture. The source labeler tags inputs by origin, the LLM compiler proposes candidate leases, and the deterministic checker (TCB) validates field proofs before any side effect commits. Enforcement hooks lower accepted leases to the MCP gateway and OS-level enforcer. Denied candidates return to the compiler.

are always runtime environment, and Skill/MCP metadata is labeled at load time. Inputs with ambiguous provenance (e.g., documents pasted by the user) require additional classification, which the current prototype handles conservatively by denying authority fields from ambiguous sources. At the OS level, this labeling maps to process-level isolation and IPC channel tagging, so each source's provenance is rooted in the process that produced it. The intent issuer extracts the user's maximum authority from structured fields in the user message, such as selected files, named destinations, and explicit approvals. The compiler takes this structured intent and the active schemas as input and emits candidate leases whose fields are populated from the labeled sources. Candidate leases can only narrow the user's authority, never widen it. The compiler may select a subset of authorized destinations or request a shorter budget, but it cannot add destinations, raise approval scope, or grant fields that the user intent does not cover.

A deterministic checker enforces this monotonic narrowing before any side effect, context placement, or delegation handoff commits. Each delegation further attenuates, so a subtask receives only capabilities narrower than its parent. The checker exposes a single transition, $\texttt{check\_and\_consume}(e, \textit{lease}, \textit{proofs}, \sigma) \rightarrow \texttt{allow}(\sigma') \mid \texttt{deny}$, that adapters must call before any side effect commits. Enforcement operates at two layers. At the tool level, boundary adapters intercept MCP calls and validate that each argument field comes from its owner source before the call executes. At the OS level, accepted leases compile to ActPlane [18] file, exec, and network policies enforced via eBPF.

A field is a named slot in a capability lease (e.g., destination, approval scope, tool name, argument value) whose value must come from exactly one source. The four context sources each contribute specific fields. *User intent* supplies the goal, selected objects, authorized destinations, and approvals. *Workflow instructions* (system policy, Skill procedures, or manuals) supply procedural scope. *Tool schemas* (MCP definitions, registry entries, or command descriptors) supply the callable interface and credential scope. *Runtime environment* (tool results, file state, script outputs) supplies observed values. No source can fill another's fields. A tool schema cannot authorize a destination, runtime environment cannot supply procedural authority, and a Skill cannot widen approval scope.

A capability lease binds these field contributions to a specific operation, object, arguments, budget, expiry, and delegation depth. Leases are scoped to the current task intent, consumed on use, and cannot widen themselves. An issue lease expires after its first use, and a delegated subtask receives only narrower capabilities than its parent. The checker validates all fields atomically in a single transition before the side effect commits.

4 Preliminary Evaluation

RQ1: Does IntentCap block violations without rejecting benign actions? IntentCap produces 0 unsafe accepts across 3,746 security-sensitive events replayed from AgentDojo, MCPTox, InjecAgent, and tau2-bench [2, 12, 16, 17]. On the utility side, IntentCap covers all 3,813 benign reference actions in the tau2-style proxy and passes 2,554 of 2,556 applicable ground-truth checks.

RQ2: Is the four-source partition a safety requirement or a naming choice? The partition is necessary. Collapsing all four sources into one generic trusted context causes 3,593 false accepts out of 3,823 events that the checker should deny (94%). Each pairwise collapse is independently necessary. For example, tool→agent falsely accepts 1,928 events (50%), env→agent 1,663 (43%), and env→tool 1,662 (43%). On 7 crafted multi-step workflow scenarios, a policy DSL that checks predicates without field ownership falsely accepts 7/7, and splitting state across independent guards falsely accepts 5/7.

RQ3: Does the same commit API work beyond tool calls? The `check_and_consume` transition generalizes across five enforcement points, including local env side effects, context placement, Skill instruction slots, delegation handoffs, and an OS monitor replay backend. Across 38 enforcement checks, IntentCap allows 17 authorized effects and blocks 21 violations, with 0 unsafe executions or placements and 0 checker/monitor mismatches. This demonstrates functional correctness across enforcement points.

RQ4: Are leases auditable authority objects? Across 24 manually labeled leases covering InjecAgent, MCPTox, and tau2 samples, all IntentCap policies match the labeled authority scope. Every non-IntentCap baseline over-grants authority on at least some of the 144 scored policy entries.

5 Discussion

IntentCap draws on capability-based security [4], least privilege [10], and information flow control [3], but addresses a problem those systems do not face. In an agent, the capability cannot be declared by a programmer at compile time, as it can with Capsicum [13] file descriptors or seccomp filters. It must be composed at runtime from context sources that share a single neural planning channel, where any source can impersonate any other, a confused deputy scenario [6] in which the deputy is a neural planner. CaMeL separates data from control to defeat prompt injections by design [1], while IsolateGPT [15] and Progent [11] restrict which operations an agent may perform through execution isolation and policy languages, respectively. At lower layers, EIM [19] and ActPlane [18] enforce constraints at the tool and OS level, and SkillGuard [9] governs what a Skill package may access. Unlike these

systems, IntentCap asks which future decisions a given context source may influence, under a given user intent, and with what proof.

References

[1] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. arXiv preprint arXiv:2503.18813. doi:10.48550/arXiv.2503.18813

[2] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*. doi:10.48550/arXiv.2406.13352

[3] Dorothy E. Denning. 1976. A Lattice Model of Secure Information Flow. *Commun. ACM* 19, 5 (1976), 236–243. doi:10.1145/360051.360056

[4] Jack B. Dennis and Earl C. Van Horn. 1966. Programming Semantics for Multiprogrammed Computations. *Commun. ACM* 9, 3 (1966), 143–155. doi:10.1145/365230.365252

[5] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*. doi:10.1145/3605764.3623985

[6] Norm Hardy. 1988. The Confused Deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review* 22, 4 (1988), 36–38. doi:10.1145/54289.871709

[7] Model Context Protocol. 2025. Specification. https://modelcontextprotocol.io/specification/2025-06-18.

[8] OpenAI. 2026. Agent Skills. https://developers.openai.com/codex/skills. Accessed: 2026-07-02.

[9] Shidong Pan, Xiaoyu Sun, Tianyi Zhang, Dianshu Liao, Meixue Si, and Zhenchang Xing. 2026. SkillGuard: A Permission Framework for Agent Skills. arXiv preprint arXiv:2606.03024. doi:10.48550/arXiv.2606.03024

[10] Jerome H. Saltzer and Michael D. Schroeder. 1975. The Protection of Information in Computer Systems. *Proc. IEEE* 63, 9 (1975), 1278–1308. doi:10.1109/PROC.1975.9939

[11] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Programmable Privilege Control for LLM Agents. arXiv preprint arXiv:2504.11703. doi:10.48550/arXiv.2504.11703

[12] Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiangyang Li. 2025. MCPTox: A Benchmark for Tool Poisoning Attack on Real-World MCP Servers. arXiv preprint arXiv:2508.14925. doi:10.48550/arXiv.2508.14925

[13] Robert N. M. Watson, Jonathan Anderson, Ben Laurie, and Kris Kennaway. 2010. Capsicum: Practical Capabilities for UNIX. In *Proceedings of the 19th USENIX Security Symposium (USENIX Security '10)*. USENIX Association, Washington, DC, USA, 29–46. https://www.usenix.org/conference/usenixsecurity10/capsicum-practical-capabilities-unix

[14] Simon Willison. 2023. The Dual LLM Pattern for Building AI Assistants That Can Resist Prompt Injection. https://simonwillison.net/2023/Apr/25/dual-llm-pattern/.

[15] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. 2025. IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society, San Diego, CA, USA. doi:10.14722/ndss.2025.241131

[16] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv preprint arXiv:2406.12045. doi:10.48550/arXiv.2406.12045

[17] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated LLM Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. 10471–10506. doi:10.18653/v1/2024.findings-acl.624

[18] Yusheng Zheng, Tianyuan Wu, Quanzhi Fu, Tong Yu, Wenan Mao, Tao Ma, Dan Williams, Wei Wang, and Andi Quinn. 2026. ActPlane: Programmable OS-Level Policy Enforcement for Agent Harnesses. arXiv preprint arXiv:2606.25189. doi:10.48550/arXiv.2606.25189

[19] Yusheng Zheng, Tong Yu, Yiwei Yang, Yanpeng Hu, Xiaozheng Lai, Dan Williams, and Andi Quinn. 2025. Extending Applications Safely and Efficiently. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI '25)*. USENIX Association. https://www.usenix.org/conference/osdi25/presentation/zheng-yusheng